Kotlin Multiplatform

Questions 1–30 with Answers

Part A — What is Kotlin Multiplatform (1–15)

Question 1

Contrast KMP with Flutter and React Native. What does Flutter render through, what does React Native talk to across, and what does Kotlin code become on iOS?

Answer: Kotlin Multiplatform (KMP) shares Kotlin code—often business logic, networking, and data models—while allowing each platform to keep its native UI. Flutter generally renders its UI through its own rendering engine and widget system. React Native lets JavaScript/TypeScript code communicate with native platform components through its runtime/bridging architecture. KMP Kotlin code for iOS is compiled to native code via Kotlin/Native and packaged for use by iOS apps.

Question 2

Map each target to the way Kotlin reaches it: Android, iOS, browser (two options), desktop, server. Name the compiler technology used for iOS.

Answer: Android: Kotlin/JVM, producing JVM bytecode that runs on Android's runtime (ART). iOS: Kotlin/Native, producing native binaries. Browser: Kotlin/JS, which compiles to JavaScript, or Kotlin/Wasm, which compiles to WebAssembly for supported environments. Desktop: commonly Kotlin/JVM, packaged with a desktop UI framework such as Compose Multiplatform. Server: commonly Kotlin/JVM on a JVM server framework. The iOS compiler technology is Kotlin/Native.

Question 3

Classify these items as shared (commonMain) or kept native, and justify briefly: Ktor API client, biometric login, input validation, push notifications, SQLDelight storage, deep-link navigation.

Answer: Ktor API client — usually shared in commonMain when configured with multiplatform engines. Input validation — shared, because rules can be platform-independent. SQLDelight storage — schemas and shared database logic can be common, while drivers are platform-specific. Biometric login — native/platform-specific API, exposed through an abstraction if shared code needs it. Push notifications — platform-specific integration, with shared business logic where useful. Deep-link navigation — usually platform-specific UI/navigation integration, although route models and parsing rules can be shared.

Question 4

Complete the expected/actual setup so each platform returns its own Platform object. Write the missing declarations for Android, iOS and desktop (JVM).

Answer: In commonMain, declare the expected function and type:

```
expect class Platform { val name: String }
expect fun getPlatform(): Platform
```

Then provide matching `actual` declarations in each platform source set, for example:

```
actual class Platform actual constructor() { actual val name = "Android" }
actual fun getPlatform() = Platform()
```

Use equivalent implementations in iosMain and jvmMain, changing the name to "iOS" and "Desktop". The expect/actual signatures must match; exact constructor syntax should follow the project's Kotlin version and declaration style.

Question 5

Find and fix the error: commonMain has expect fun currentTimeMillis(): Long { return 0L }.

Answer: An `expect` declaration has no implementation body in commonMain. Write:

```
expect fun currentTimeMillis(): Long
```

Then implement it with an `actual` function for each target. For example, JVM can use `System.currentTimeMillis()`; platform-specific implementations should use APIs supported by their targets.

Question 6

A project targets androidTarget(), jvm(), iosArm64() and iosSimulatorArm64(). An expect declaration in commonMain has actual implementations only in androidMain and jvmMain. State what fails and give two ways to fix it.

Answer: The iOS compilations fail because the expected declaration has no matching actual implementation for the iOS targets. Fix it by (1) adding a compatible actual implementation in iosMain (shared by the configured iOS targets through the hierarchy), or (2) redesigning the code to avoid expect/actual—for example, inject a platform-specific implementation through an interface or constructor. Every target compiling the expect declaration must receive a valid implementation.

Question 7

Explain why the compiler merges when building for iosArm64. Why is code in appleMain visible to iOS targets but not androidMain?

Answer: In the default KMP hierarchy, related Apple targets can share an intermediate source set such as appleMain. The iosArm64 compilation includes source sets along its dependency path, including appleMain and commonMain. androidMain is on a separate branch and does not depend on appleMain, so Apple-specific declarations are not visible to Android. This is source-set hierarchy, not automatic visibility across all targets.

Question 8

Write a function in iosMain that returns UIDevice.currentDevice.systemVersion and explain why it cannot live in commonMain.

Answer: In iosMain, using Kotlin/Native platform APIs:

```
import platform.UIKit.UIDevice
fun iosSystemVersion(): String = UIDevice.currentDevice.systemVersion
```

This cannot be placed directly in commonMain because UIKit and UIDevice are iOS-specific APIs unavailable to all KMP targets. If common code needs the value, expose it through expect/actual or dependency injection.

Question 9

Write the sourceSets block that adds ktor-client-core to commonMain, the OkHttp engine to androidMain and the Darwin engine to iosMain.

Answer:

```
sourceSets {
commonMain.dependencies {
implementation("io.ktor:ktor-client-core:$ktorVersion")
}
androidMain.dependencies {
implementation("io.ktor:ktor-client-okhttp:$ktorVersion")
}
iosMain.dependencies {
implementation("io.ktor:ktor-client-darwin:$ktorVersion")
}
}
```

Define ktorVersion in the project's version catalog or build configuration. Keep Ktor module versions aligned.

Question 10

State the difference between Kotlin Multiplatform and Compose Multiplatform. Describe the module layout for shared logic with native UI on Android and iOS.

Answer: Kotlin Multiplatform is the technology for sharing Kotlin code across platforms. Compose Multiplatform is a UI framework that lets developers share Compose-based UI on supported platforms. For native UI, use a shared KMP module for business logic and data, an Android app using Jetpack Compose or Views, and an iOS app using SwiftUI/UIKit. Both apps call the shared module's APIs, while each keeps its native presentation layer.

Question 11

Name the stability level of iOS, Kotlin/Wasm, expect/actual classes, and Compose Multiplatform for web. Decide whether each should be used in a production app today and justify.

Answer: Stability changes by Kotlin, library, and framework release, so check the exact versions used in the lecture. Kotlin/Native iOS support is established and is commonly suitable for production. Kotlin/Wasm and Compose Multiplatform web have had evolving maturity and may be experimental or beta depending on the release; verify current release status before committing to them. expect/actual is a language mechanism, not a separate platform stability level, and is suitable when used correctly. Production decisions should consider official status, target-browser support, library compatibility, and team experience.

Question 12

Write a commonTest test for fun isValidEmail(text: String): Boolean using kotlin.test. Include one valid and one invalid case, and explain how to run it on JVM and iOS simulator.

Answer:

```
import kotlin.test.Test
import kotlin.test.assertTrue
import kotlin.test.assertFalse

class EmailValidationTest {
@Test fun validEmailIsAccepted() {
assertTrue(isValidEmail("user@example.com"))
}
@Test fun invalidEmailIsRejected() {
assertFalse(isValidEmail("not-an-email"))
}
}
```

Put the test in commonTest. Run the JVM test task (commonly `./gradlew jvmTest`) and the configured iOS simulator test task, whose exact name depends on the targets and Gradle plugin setup (inspect `./gradlew tasks --all`).

Question 13

List two limitations of KMP in 2026—Swift interop, binary size, and team skills—and propose a mitigation for each.

Answer: Swift interop: some Kotlin APIs can be awkward to consume from Swift; keep the exported API small and Swift-friendly, and provide wrappers where needed. Binary size: native frameworks and dependencies can increase app size; remove unused dependencies, inspect release builds, and avoid unnecessary framework exports. Team skills: developers need Kotlin, platform-specific APIs, and build tooling knowledge; provide onboarding, coding guidelines, and ownership for native integration.

Question 14

Write a LoginValidator class in commonMain (email and password rules) and show how it is consumed from an Android ViewModel and from a SwiftUI view model.

Answer: Shared validator:

```
class LoginValidator {
fun isValidEmail(email: String) =
email.contains("@") && email.substringAfter("@").contains(".")
fun isValidPassword(password: String) = password.length >= 8
}
```

Android ViewModel can create/use `LoginValidator()` and call its methods before submitting credentials. In SwiftUI, import the generated shared framework/module, create a wrapper view model, and call the exported validator methods. Exact Swift names depend on framework naming and exported API. This example illustrates basic validation only; production email validation and authentication need stronger handling.

Question 15

An existing Android-only app needs an iOS version. Order the layers for migration into shared code and justify the first choice: data models, networking, local storage, business rules, UI.

Answer: A practical order is: (1) data models and pure business rules, (2) networking/API client, (3) local storage behind a multiplatform abstraction and with compatible drivers, then (4) decide which UI elements to share. Start with models and business rules because they are usually platform-independent and have fewer platform dependencies. Keep UI native if the goal is to use Jetpack Compose on Android and SwiftUI on iOS.

Part B — KMP Project Configuration (16–30)

Question 16

Compare a single shared module with several shared modules: one advantage and one drawback each. State the signal that tells a team to split.

Answer: Single module advantage: simple setup and dependency management. Drawback: it can become large and tightly coupled. Several modules advantage: clear boundaries and selective reuse. Drawback: more Gradle configuration and dependency coordination. Split when the module has distinct domains, different ownership, excessive build times, or consumers need different subsets of functionality.

Question 17

Write the settings.gradle.kts include lines for a project with :shared, an Android app and a desktop app, and show the dependency line each app uses to consume :shared.

Answer:

```
// settings.gradle.kts
include(":shared", ":androidApp", ":desktopApp")
```

In each app's build.gradle.kts, add the dependency in the appropriate dependency block, for example:
`dependencies { implementation(project(":shared")) }`

For a multiplatform module, ensure the shared module exposes a compatible variant/API for each consumer. Desktop and Android may need target-specific configuration depending on the module setup.

Question 18

Two feature modules that both depend on the same library are exposed as two separate iOS frameworks. Explain what ends up inside the app and describe the concrete failure with shared state.

Answer: Depending on framework export and linking settings, each framework may bundle or link the shared library separately. If each framework contains its own copy of a singleton or cache, updates made through one framework may not be visible through the other. This can cause inconsistent state, such as one feature seeing a logged-in user while another sees no user. Consolidate the shared dependency into one framework or configure framework linking so the dependency is not duplicated; verify the actual binary/linkage setup.

Question 19

Write the build.gradle.kts of an umbrella module that depends on :feature:auth and :feature:profile and produces one static iOS framework named Shared exporting both.

Answer: Example KMP configuration (adapt plugin and target names to the project):

```
kotlin {
val iosTargets = listOf(iosArm64(), iosSimulatorArm64())
iosTargets.forEach { target ->
target.binaries.framework {
baseName = "Shared"
isStatic = true
export(project(":feature:auth"))
export(project(":feature:profile"))
}
}
sourceSets {
commonMain.dependencies {
api(project(":feature:auth"))
api(project(":feature:profile"))
}
}
}
```

Framework export makes dependency APIs available through the generated framework. Use the project's actual plugin versions and target DSL.

Question 20

Swift cannot see the types from :feature:auth. Find and fix the cause in the shown framework configuration.

Answer: The common causes are that the auth module is only an implementation dependency, is not exported by the framework, or the framework configuration is applied to the wrong targets. Declare the dependency in the umbrella module and export it in the framework binary:

```
commonMain.dependencies { api(project(":feature:auth")) }
target.binaries.framework { export(project(":feature:auth")) }
```

Then rebuild the framework and verify that Swift imports the correct generated framework/module. Also confirm that the auth module is a compatible KMP module and its API is exportable.

Question 21

Explain why a Kotlin dependency cannot be shared across two separately built iOS frameworks, and why one umbrella framework is the only way to guarantee a single copy.

Answer: Two separately built frameworks can each statically embed the same dependency, producing separate copies of its code and state or duplicate symbols. A single umbrella framework centralizes the shared dependency and can prevent independently embedding it in each feature framework. It is not literally the only possible technical design—dynamic frameworks or careful linkage can also avoid duplication—but one umbrella framework is a straightforward way to centralize the exported KMP API and dependency graph.

Question 22

For a monorepo, a two-repository layout (Android + shared | iOS) and a three-repository layout (Android | iOS | shared), state one advantage and one drawback of each, and how versions are used.

Answer: Monorepo: easy atomic changes across all code; drawback is repository/build complexity. Shared code can be changed alongside apps, though releases may still be versioned. Two repositories: Android and shared code can change together; drawback is coordination with iOS. iOS consumes a published or otherwise distributed compatible shared version. Three repositories: independent ownership and release cycles; drawback is more versioning and integration work. Both apps update to compatible shared releases.

Question 23

Three repositories: the shared code must also be published for Android/JVM, not only iOS. Give two publishing options and the trade-off between them.

Answer: Option 1: publish a JVM artifact to a Maven-compatible repository (such as an internal Maven repository or an approved artifact registry). This is convenient for Gradle dependency management and versioning, but requires repository access and publication setup. Option 2: publish a versioned package/artifact through an internal artifact store or Git-based package registry. This may fit existing company infrastructure, but access control, retention, and compatibility management still need configuration. An iOS framework artifact alone is not sufficient for Android/JVM consumers.

Question 24

Configure maven-publish in the shared module to publish to mavenLocal() with group uz.tpp.shared and version 1.0.0. Write the dependency line the Android repository uses to consume it.

Answer: A simplified example is:

```
group = "uz.tpp.shared"
version = "1.0.0"
publishing {
repositories { mavenLocal() }
}
```

Apply the `maven-publish` plugin and configure publication for the intended JVM/Android variant. For a multiplatform library, publication setup depends on the Kotlin Gradle plugin and target configuration; ensure a JVM-compatible publication is generated. In the consuming Android repository, configure `mavenLocal()` in repositories and use:

```
implementation("uz.tpp.shared:shared:1.0.0")
```

The artifact name depends on the publication's configured artifact ID.

Question 25

Configure Gradle to build an XCFramework named Shared for iosArm64 and iosSimulatorArm64, and write the .binaryTarget entry of a Swift package that consumes it from a URL. Explain why a checksum is required.

Answer: In a KMP module, create an XCFramework and add both device and simulator framework binaries. A common pattern is:

```
val xcframework = XCFramework("Shared")
listOf(iosArm64(), iosSimulatorArm64()).forEach { target ->
target.binaries.framework {
baseName = "Shared"
isStatic = true
xcframework.add(this)
}
}
```

Swift Package manifest entry:

```
.binaryTarget(name: "Shared", url: "https://example.com/Shared.xcframework.zip", checksum:
"<computed-checksum>")
```

Compute the checksum with Swift Package Manager's `swift package compute-checksum Shared.xcframework.zip`. The checksum lets SwiftPM verify the downloaded binary artifact has not changed unexpectedly. Replace the example URL and checksum with the real release values.

Question 26

Compare local (source) and remote (artifact) distribution on four axes: iOS developer onboarding, iteration speed, versioning overhead, and who can edit shared code.

Answer: Local/source distribution: onboarding can be simple in a monorepo, iteration is fast because developers can change shared code directly, versioning overhead is lower for coordinated changes, and developers with repository access can edit the shared source. Remote/artifact distribution: onboarding requires repository credentials and dependency setup, iteration may be slower due to publish/update cycles, explicit versions and releases are needed, and only the shared-library maintainers or contributors with source access can edit it.

Question 27

Describe the hybrid workflow for an iOS developer who must change shared code but normally consumes a published artifact. Include checkout, local dependency, verification, publishing, switching back, and the correct order of publish and app update.

Answer: (1) Check out the shared repository and the iOS application repository. (2) Temporarily configure the iOS app to consume the local framework or local package build instead of the published artifact. (3) Make the shared-code change and run shared tests plus relevant iOS tests. (4) Build and verify the framework in the app. (5) Commit and push the shared change, then publish a versioned artifact/framework. (6) Update the iOS app to consume that published version and verify the app again. (7) Remove the temporary local dependency configuration. The correct order is to publish the verified shared artifact first, then update the app to reference that released version.

Question 28

Two apps need shared libraries: App A uses logging and networking; App B uses networking and analytics. Draw the repository graph and state what each app gets from its own umbrella.

Answer:

```
logging ----\
appA-umbrella ---- App A
networking -/

networking -\
appB-umbrella ---- App B
analytics --/
```

App A's umbrella depends on and exports logging plus networking. App B's umbrella depends on and exports networking plus analytics. Each app receives only the APIs and dependencies its own umbrella exposes. Avoid embedding separate copies of networking within one app if shared runtime state or duplicate symbols are a concern.

Question 29

Choose a configuration (modules, repositories, distribution) for each case: a solo developer with a small app; four engineers releasing Android and iOS together; separate Android and iOS teams with independent release cycles.

Answer: Solo developer with a small app: one repository and one shared module; local source dependency is simplest. Four engineers releasing Android and iOS together: a monorepo with a shared module or a few clear feature modules supports coordinated changes and atomic pull requests. Separate Android and iOS teams with independent release cycles: keep platform apps separate and publish the shared module as a versioned artifact/framework, with explicit compatibility and release rules.

Question 30

A single :shared module has grown to 40k lines with slow builds. Describe step by step how to split it into feature modules plus an umbrella module without changing the API the iOS app uses.

Answer: (1) Record the current public API and add tests around it. (2) Identify domain boundaries and move code into feature modules, keeping package names and public signatures stable where possible. (3) Update internal dependencies so feature modules depend on lower-level core modules without circular dependencies. (4) Create an umbrella module that depends on and exports the feature modules. (5) Generate the iOS framework with the same public framework name and compatible exported APIs. (6) Update the build so the iOS app consumes the umbrella framework rather than each feature framework. (7) Compare generated APIs, run common/JVM/Android/iOS tests, and validate binary linking and shared state. (8) Migrate gradually and keep compatibility wrappers for APIs that moved. (9) Measure build times and remove obsolete dependencies only after validation.

Note: Kotlin Multiplatform Gradle DSL and platform maturity can vary by Kotlin/plugin version. Treat code snippets as templates and verify target names, plugin versions, and publication configuration in your project.